The first thing for us to do is to define a mean function class:

```
class QuadraticMean(gpytorch.means.Mean):
    def __init__(self, batch_shape=torch.Size(), bias=True):
        ...  ❶

    def forward(self, x):
        ...  ❶
```

❶ To be discussed shortly.

This class extends the `gpytorch.means.Mean` class, which is the base for all GPyTorch mean function implementations. To implement our custom logic, we need to rewrite two methods `__init__()` and `forward()`—exactly the same as when we implement a GP model!

In `__init__()`, we need to declare what parameters our mean function contains; this is called *parameter registration*.

While a linear function has two parameters, a slope and an intercept, a quadratic function has three: a coefficient for the second-order term x^2, a coefficient for the first-order term x, and one for the zeroth-order term which is typically called the bias. This is illustrated in Figure 3.9.

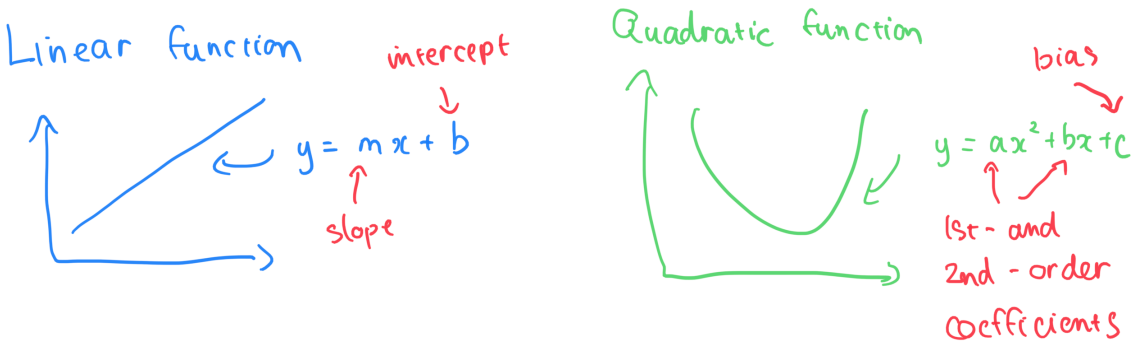


Figure 3.9 The functional forms of a linear function and a quadratic function. The linear function has two parameters, while the quadratic function has three. When these functions are used as a GP's mean function, the corresponding parameters are the GP's hyperparameters.

With that in mind, we implement the `__init__()` method like so:

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>